Reason. Plan. Act. Physical AI Workshop

Lab 1.2: Motion Planning part 1

Guy Azran and Sarah Keren

The Taub Faculty of Computer Science
Technion - Israel Institute of Technology



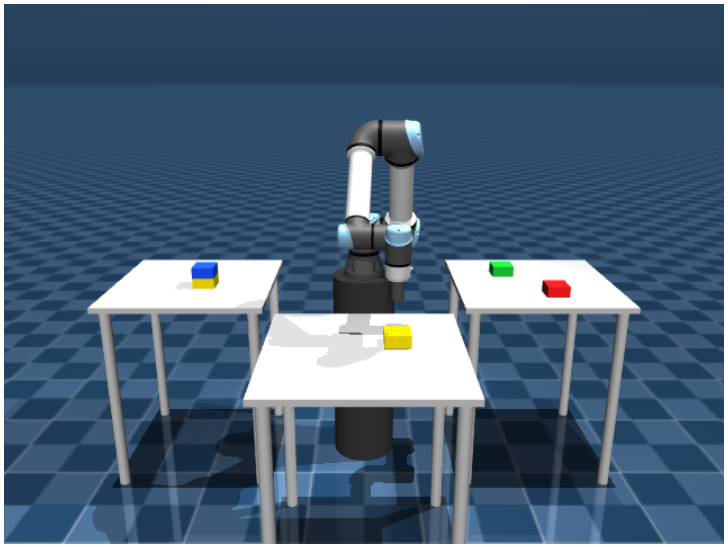
From symbols to joint angles

Guy Azran and Sarah Keren

Lab 1.1 ended with `pick-up(b3, t2)`.

A robot arm does not understand `pick-up`. It understands **joint angles**.

Which joint angles bring the suction cup to the block?

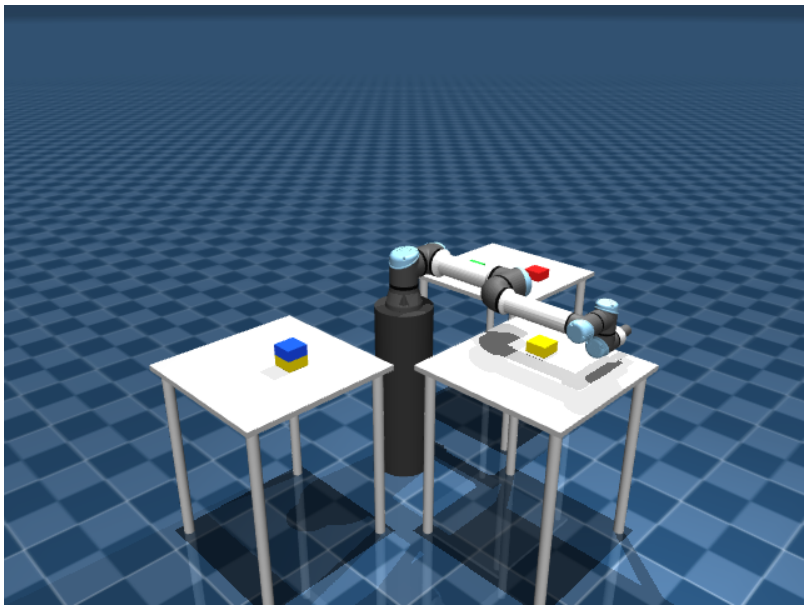


1.1 Task planning	2.1 Motion planning	3.1 State estimation	4.1 Integrated TAMP
1.2 Inverse kinematics	2.2 Control	3.2 Decoupled TAMP	4.2 VLAs

Today

Learn	The NTBW in MuJoCo: states, workspace, configuration space
Do	Task 1: how close do the servos get?
Do	Task 2: find a configuration by hand
Learn	Forward kinematics, numerical inverse kinematics
Do	Tasks 3 and 3.1: a damped least squares IK solver
Learn	Analytical IK and reference frames
Do	Task 4: from the world frame to the robot's base frame
Learn	Grounding `pick-up` with IK
Do	Task 5: put the block down
Learn	Straight lines in configuration space
Do	Task 6 and the bonus: check the line, move straight in the workspace

The N-Table Blocks World in MuJoCo



- Scene in MJCF (XML): three tables, five blocks, named as in the PDDL problem
- The UR5e arm from MuJoCo Menagerie, attached with `MjSpec`
- A suction cup: grasping turns on a **weld constraint**
- `tcp`: the tool center point, the point we position

The state of the world

Guy Azran and Sarah Keren

```
print("qpos has", model.nq, "numbers")
```

```
qpos has 41 numbers
  robot_shoulder_pan_joint     0.. 0
  robot_shoulder_lift_joint    1.. 1
  robot_elbow_joint            2.. 2
  robot_wrist_1_joint          3.. 3
  robot_wrist_2_joint          4.. 4
  robot_wrist_3_joint          5.. 5
  b1_joint                     6..12
  b2_joint                    13..19
  b3_joint                    20..26
  b4_joint                    27..33
  b5_joint                    34..40
```

A state is a vector of numbers, `data.qpos`:

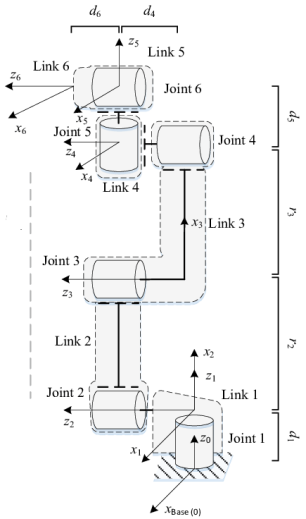
- 6 joint angles of the robot
- 7 numbers per block: position and orientation (a quaternion)

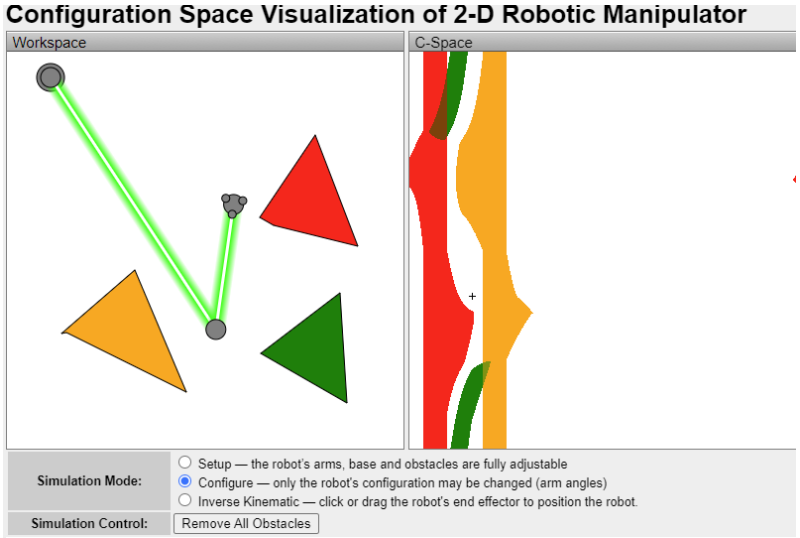
Where is the fact `(on b2 b1)`? Nowhere. It is implied by the numbers, and we will have to compute it (state estimation, Lab 3.1).

Workspace and configuration space

Guy Azran and
Sarah Keren

- **Workspace**: the physical space the robot moves in. Tasks live here: “the cup 10 cm above **b3**”
- **Configuration space**: all configurations $q = (q_1, \ldots, q_6)$ of the robot. The robot is controlled here

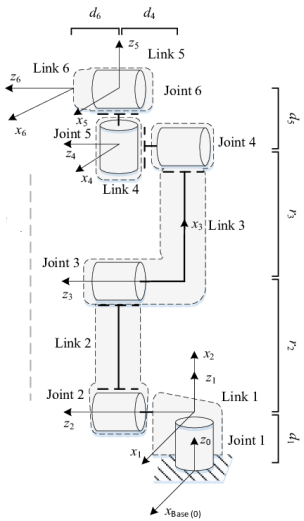


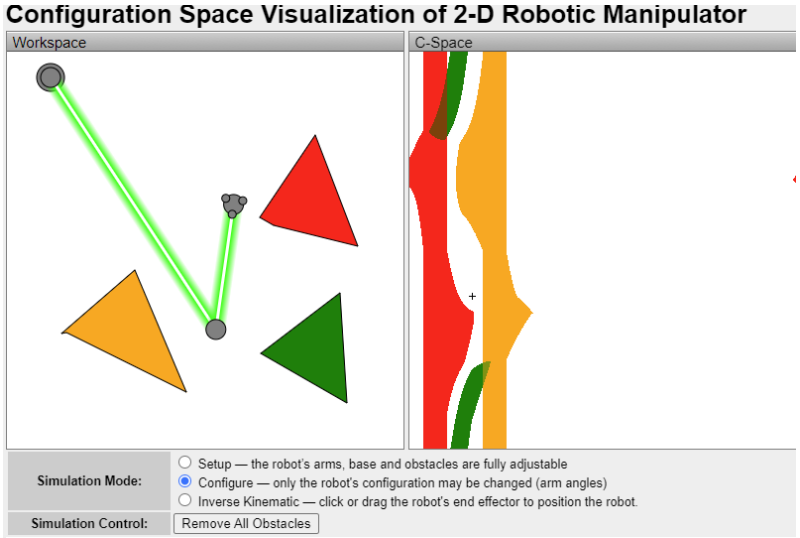


How many dimensions does the C-space of the whole scene have? Why does `qpos` have more numbers?

Workspace and configuration space

Guy Azran and
Sarah Keren

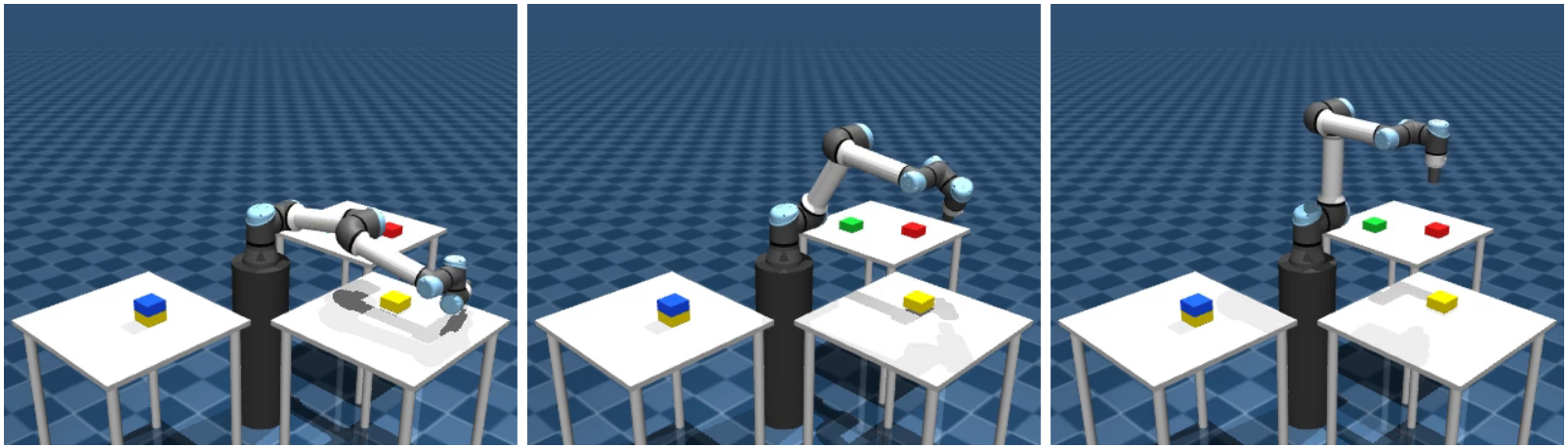




Try it:

`https://robotics.cs.unc.edu/C-space/index.html`

Moving the robot



- Our **actions are target configurations**: `data.ctrl[ctrl_idx] = target`
- Position servos push each joint toward its target
- `mj_step` advances the physics by 2 ms. We keep the target applied, and physics decides what happens on the way

Task 1: how close do the servos get?

Guy Azran and
Sarah Keren

⚒ **Task 1**: Change `number_of_steps` to 50 and to 2000 and re-run the cell. How close does the robot get to the target in each case? Does it ever get there exactly?

```
target = np.array([0, -np.pi / 2, np.pi / 2, -np.pi / 2, -np.pi / 2, 0])
number_of_steps = 500

frames = []
for i in range(number_of_steps):
    data.ctrl[ctrl_idx] = target  # the action: a target configuration
    mujoco.mj_step(model, data)
```

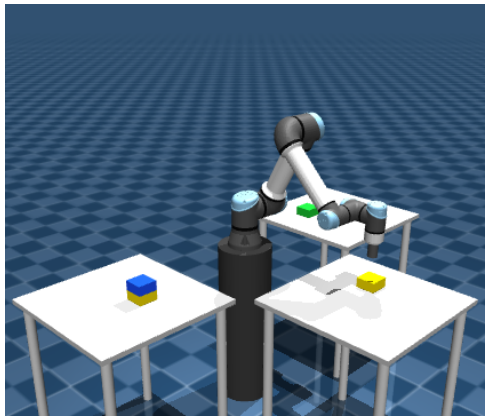
- Compare the printed target and configuration
- Which joints stop short? We will see why in the control lab (Lab 2.2)

Task 2: find a configuration by hand

Guy Azran and
Sarah Keren

⚒ **Task 2**: Recall that our task is to pick up and move blocks. Find, by hand, a configuration that brings the suction cup right above the yellow block `b3` on table `t2` (the one in front of the robot, along the x axis), with the cup pointing down. Aim for the `tcp` site to be about 10 cm above the top of the block. Give it 3-5 attempts, using the printed distance and the video to guide you. The cell prints the position of the tool center point and its distance from the target.

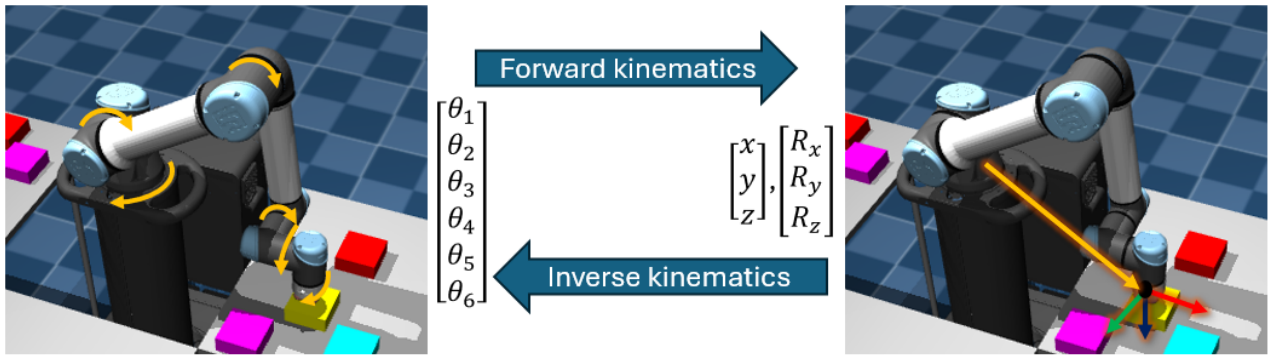
```
reset()
your_config = HOME.copy()  # your code here: edit the
configuration
```



- Joints: shoulder pan, shoulder lift, elbow, wrist 1, 2, 3
- Start from `HOME` and change one joint at a time
- 3–5 attempts are enough

Forward and inverse kinematics

Guy Azran and
Sarah Keren



Forward: a chain of homogeneous transformations $T = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix}$

$$T_{\text{tcp}}(q) = T_{\text{base}}\, T_1(q_1)\, T_2(q_2) \cdots T_6(q_6)\, T_{\text{tool}}$$

MuJoCo computes it in `mj_forward`.

Inverse is harder: the map is nonlinear, and a pose may have no solution, several, or infinitely many.

Numerical IK with the Jacobian

Guy Azran and
Sarah Keren

Near q, the pose changes linearly:

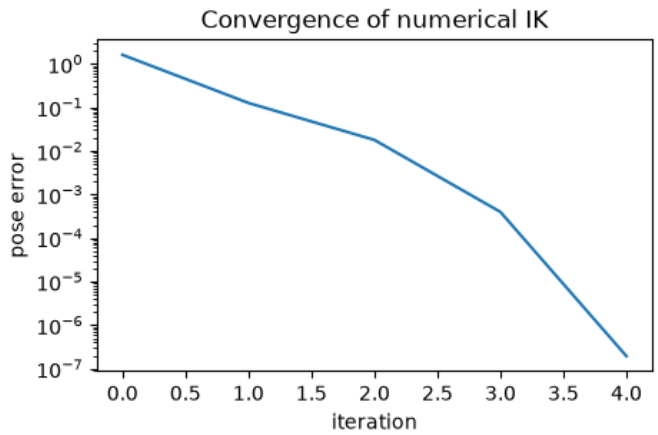
$$\Delta x \approx J(q)\,\Delta q$$

Iterate from an initial guess:

1. error $e = x_{\text{target}} - f(q)$
2. step $\Delta q = J^{-1}e$
3. $q \leftarrow q + \Delta q$

Near singular configurations J^{-1} explodes. **Damped least squares**:

$$\Delta q = J^{\top}(JJ^{\top} + \lambda^2 I)^{-1}e$$



Works for any robot, finds one solution (near the guess), may not converge.

Task 3: damped least squares

Guy Azran and
Sarah Keren

⚒ **Task 3**: Complete the update step of `numerical_ik` below with damped least squares.

Hints: `np.linalg.solve(A, b)` solves $Ay = b$, `J.T` transposes, and `np.eye(6)` is the identity.

```
    for iteration in range(max_iterations):
        set_configuration(q)
        error = pose_error(target_position, target_rotation)
        errors.append(np.linalg.norm(error))
        if errors[-1] < tolerance:
            converged = True
            break

        J = tcp_jacobian()
        delta_q = np.zeros(6)  # your code here: damped least squares step

        q = wrap_angles(q + delta_q)
```

It should converge within a handful of iterations.

Task 3.1: random starts and unreachable targets

Guy Azran and
Sarah Keren

⚒ **Task 3.1**: Run `numerical_ik` for the same target from several random initial guesses (`np.random.uniform(-np.pi, np.pi, 6)`). How many different solutions do you find? Then try a target that is clearly out of reach, e.g. `[1.5, 0, 0.9]` . What happens to the error?

```
# your code here
```

- Do different initial guesses give different solutions?
- What does the error do when there is no solution?

Would you trust this solver inside a control loop, called thousands of times per second?

Analytical IK and reference frames

For the UR5e, IK has a closed form with up to **8 solutions**: shoulder left or right $\times$ elbow up or down $\times$ wrist flipped or not.

The `ur-analytic-ik` library wants the **flange** pose in the robot's **base** frame. We have a **tcp** pose in the **world** frame.

A frame A maps its coordinates to the world: $p_{\text{world}} = R_A\, p_A + t_A$. A pose T_B relative to A:

$$T_A^{-1}\, T_B = \begin{bmatrix} R_A^\top R_B & R_A^\top (t_B - t_A) \\ 0 & 1 \end{bmatrix}$$

Task 4: from the world frame to the base frame

Guy Azran and
Sarah Keren

⚒ **Task 4**: Implement `world_to_base`, which takes a pose (position and rotation matrix) in world coordinates and returns the same pose relative to the robot's base frame.

```python
def world_to_base(position, rotation):
    """Transform a pose from world coordinates to the robot's base frame."""
    base = data.body("mount")
    base_position, base_rotation = base.xpos, base.xmat.reshape(3, 3)

    position_in_base = position  # your code here
    rotation_in_base = rotation  # your code here

    return position_in_base, rotation_in_base
```

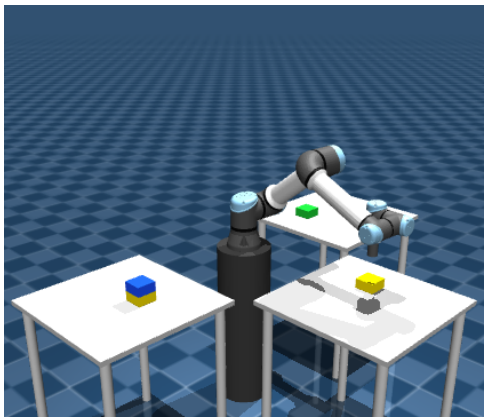
The check compares with the library's own forward kinematics: they should agree to about a millimeter.

Eight solutions. Which one?

Guy Azran and
Sarah Keren

```
reset()
solutions = analytical_ik(above_b3, FACING_DOWN)
```

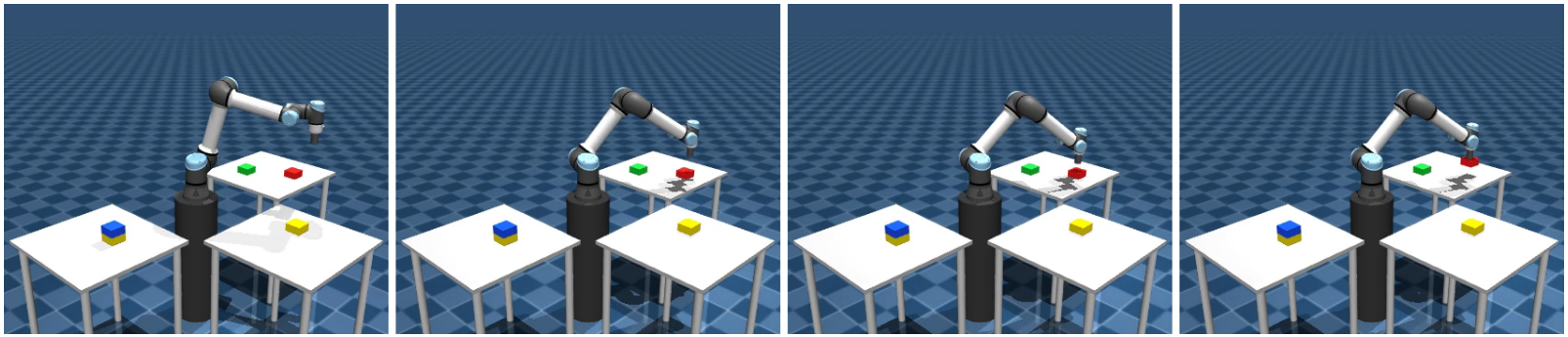
```
8 solutions:
   [-0.056 -0.668  1.079  1.161  1.571  1.515]  tcp error: 0.9 mm
   [-0.056  0.362 -1.079  2.287  1.571  1.515]  tcp error: 0.8 mm
   [-0.056 -1.068  1.811 -2.314 -1.571 -1.627]  tcp error: 0.8 mm
   [-0.056  0.641 -1.811 -0.401 -1.571 -1.627]  tcp error: 0.7 mm
   [-2.756  2.501  1.811 -2.741  1.571 -1.185]  tcp error: 0.7 mm
   [-2.756 -2.074 -1.811 -0.828  1.571 -1.185]  tcp error: 0.8 mm
   [-2.756  2.779  1.079  0.854 -1.571  1.957]  tcp error: 0.8 mm
   [-2.756 -2.473 -1.079  1.981 -1.571  1.957]  tcp error: 0.9 mm
```



- Keep the **collision-free** ones
- Pick the one **closest** to the current configuration (angles are periodic)
- 10× faster than numerical IK, all solutions at once

Grounding `pick-up(b5, t3)`

Guy Azran and
Sarah Keren



1. move to a **pre-grasp** pose 10 cm above the block
2. descend until the cup almost touches it
3. turn on the suction
4. lift

Each step: a target pose, IK, and `move_to`.

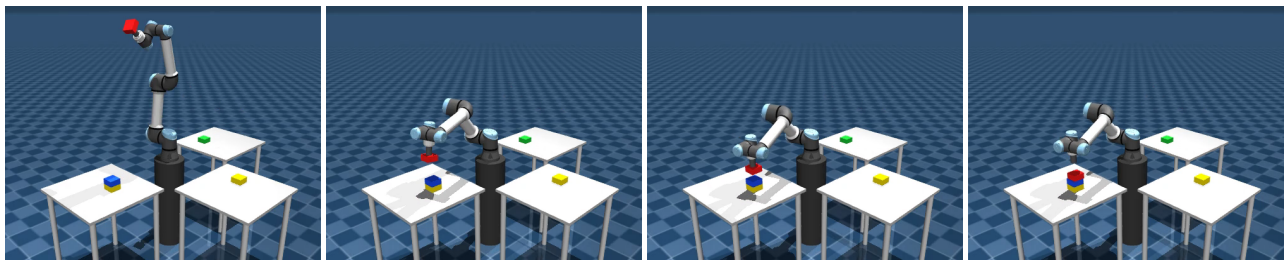
Task 5: put the block down

Guy Azran and Sarah Keren

⚒ **Task 5**: Now put it down: complete the `put-down` / `stack` part. Move the block above `b2` (the blue block on `t1`), lower it until it rests on `b2`, release the suction, and move back up. Take care with the heights: `pose_above("b2", h)` gives a pose `h` above the *top* of `b2`, and the block we are holding extends `2 * BLOCK_HALF_HEIGHT` below the cup. Leave a few millimeters of clearance so that the held block does not push into `b2` before it is released. Choose the IK solutions as in the pick-up above.

```
frames = []
# your code here

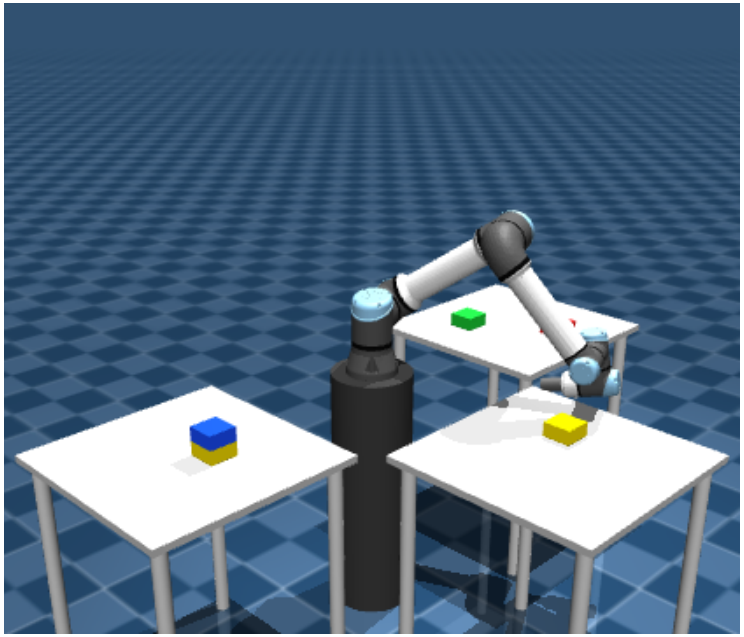
show_video(frames)
print("block positions:", get_block_positions())
```



Straight lines in configuration space

Guy Azran and
Sarah Keren

- `move_to` moves all joints at once: a **straight line in C-space**
- In the workspace, every link sweeps an arc
- Nothing checks what the arc passes through



Solution 6 above `b1` is collision-free, but the arm hits the table 22% of the way there, and stays stuck.

Task 6: check the straight line

Guy Azran and Sarah Keren

⚒ **Task 6**: Implement `path_in_collision`, which checks whether the straight line between two configurations passes through a collision (by checking a number of intermediate configurations). Then use it to find out which of the eight solutions for the pose above `b1` can be reached from `HOME` in a straight line, and verify your answer by executing one that can and one that cannot.

```
def path_in_collision(q_from, q_to, n_steps=50):
    """Whether the straight line in configuration space from
q_from to q_to passes through a collision."""
    return False  # your code here
```

- Which of the eight solutions can be reached from `HOME` in a straight line?
- Execute one that can, and one that cannot

Is there always one? Think of a tall tower in the way, or of the block in the gripper.

Bonus task: straight lines in the workspace

Guy Azran and
Sarah Keren

⚒ **Bonus task**: Straight lines in the *workspace* are often what we want (think of sliding a block along a table). Implement a "Cartesian" motion: interpolate the *pose* of the tool center point between its current pose and a target 20 cm away at the same height, solve IK for each intermediate pose, and execute the resulting configurations one after the other. Watch what happens if, for each intermediate pose, you pick `solutions[0]` instead of the solution closest to the previous configuration.

```
# your code here
```

- Interpolate the *pose*, solve IK for each intermediate pose
- Try `solutions[0]` instead of the closest solution, and watch

Get to work!

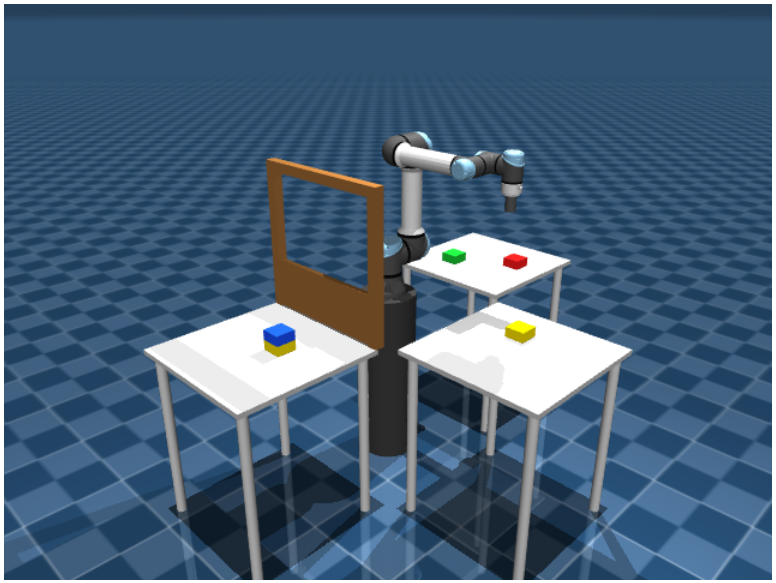
Summary

In this tutorial we

- met the NTBW in MuJoCo, where the state is a vector of joint angles and block poses, not a set of facts
- separated the workspace, where tasks are specified, from the configuration space, where the robot is controlled
- computed forward kinematics as a chain of transformations
- solved inverse kinematics numerically with the Jacobian, and analytically with all 8 solutions
- grounded `pick-up` and `stack` with IK and straight moves
- saw that a target configuration is not a safe way to get there: straight lines in C-space can collide

What comes next

Guy Azran and
Sarah Keren



A wall with a window in front of `t1`

A target configuration is not a safe way to get there.

Motion planning (Lab 2.1) finds intermediate configurations, using what we built today:

- a collision check of one configuration
- a collision check of a straight segment